

Pipelines de Integração Cotações & Salesforce

InterCement — Documentação dos notebooks Jupyter (.ipynb) do processo de ETL e consolidação de dados de cotações comerciais

ORDEM CORRETA DE LEITURA / EXECUÇÃO

1. **ETL_SalesForce_Pipeline.ipynb** — extração e limpeza da base Salesforce
2. **ETL_Cotacao_Pipeline.ipynb** — extração e limpeza da base de Cotações
3. **Concolida_Cotacao_SalesForce.ipynb** — junção das duas bases e cálculo das regras de negócio (janela de 30 dias)

Linguagem: Python 3.13 (pandas)

Ambiente: Jupyter Notebook

Data do documento: 29 de agosto de 2026

Bibliotecas utilizadas:

pandas · os · re · logging · datetime

Sumário

1. Visão geral do processo

2. Etapa 1 — ETL_SalesForce_Pipeline

- 2.1 Objetivo e arquivos
- 2.2 Fluxo de execução
- 2.3 Funções e regras de negócio

3. Etapa 2 — ETL_Cotacao_Pipeline

- 3.1 Objetivo e arquivos
- 3.2 Fluxo de execução
- 3.3 Funções e regras de negócio

4. Etapa 3 — Concolida_Cotacao_SalesForce

- 4.1 Objetivo e arquivos
- 4.2 Fluxo de execução
- 4.3 Funções e regras de negócio
- 4.4 Regra da janela móvel de 30 dias (detalhe)

5. Observações técnicas e pontos de atenção

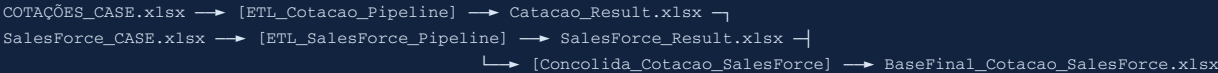
1. Visão geral do processo

O processo é composto por três notebooks Python que, em conjunto, formam um pipeline de ETL (Extract, Transform, Load) responsável por consolidar dados de cotações comerciais com informações complementares do Salesforce, aplicando regras de negócio de janela móvel de 30 dias.

1. **ETL_SalesForce_Pipeline.ipynb** lê a base bruta do Salesforce, trata tipos, remove duplicidades/linhas vazias e gera `SalesForce_Result.xlsx` .
2. **ETL_Cotacao_Pipeline.ipynb** lê a base bruta de Cotações, converte colunas decimais, trata valores vazios e gera `Catacao_Result.xlsx` .
3. **Concolida_Cotacao_SalesForce.ipynb** lê os dois arquivos de resultado gerados acima, faz um `LEFT JOIN` entre eles, aplica a regra de janela de 30 dias e exporta o arquivo final `BaseFinal_Cotacao_SalesForce.xlsx` .

Dependência entre notebooks: o terceiro notebook (Concolida_Cotacao_SalesForce) depende diretamente dos arquivos de saída gerados pelos dois primeiros. Por isso, ele deve ser executado **sempre por último**, após os arquivos `Catacao_Result.xlsx` e `SalesForce_Result.xlsx` já existirem na pasta de resultados.

Diagrama de dependência entre arquivos



ETL_SalesForce_Pipeline.ipynb

Log gerado: pipeline_salesforce.log

2.1 Objetivo

Extrai a base bruta de oportunidades/cotações do Salesforce, padroniza tipos de colunas, remove registros inválidos (linhas vazias, duplicados e tipo de cotação nulo) e exporta uma base tratada que será usada na etapa de consolidação (Etapa 3).

ENTRADA

- `SalesForce_CASE.xlsx`
(pasta *Base Dados*)

SAÍDA

- `SalesForce_Result.xlsx`
(pasta *Base Result*)

2.2 Fluxo de execução (função `pipeline_salesforce()`)

- Carregar** o arquivo Excel com tipos de coluna pré-definidos (texto/string).
- Tratar colunas de texto:** substitui vazios/NaN por `"0"` .
- Tratar colunas numéricas** (`Frete Comercial` , `Chapa`): substitui vazios/NaN por `0.0` .
- Remover linhas totalmente vazias** (todas as colunas nulas ou em branco).
- Remover duplicados** com base nas chaves `ID da Cotação do SAP` + `Material: Código do material` + `Cód. Expedição` .
- Remover linhas** onde `TipoCotação = "0"` (registros sem tipo de cotação válido).
- Filtrar condições específicas** (diagnóstico, não altera a base principal): identifica linhas com `TipoCotação` vazio e `Frete Comercial/Chapa ≤ 0`.
- Exportar** o resultado para `SalesForce_Result.xlsx` .

2.3 Funções principais

`carregar_arquivo(caminho, dtype_cols)` **EXTRACT**

Lê o Excel definindo tipo `string` para as colunas-chave (evita erros de conversão automática do pandas).

`tratar_colunas_texto(df, cols)` **TRANSFORM**

Preenche valores ausentes/vazios das colunas de texto com `"0"` .

`tratar_colunas_numericas(df, cols)` **TRANSFORM**

Preenche valores ausentes/vazios de `Frete Comercial` e `Chapa` com `0.0` e converte para `float` .

`remover_linhas_vazias(df)` **TRANSFORM**

Remove linhas em que todas as colunas estão vazias ou nulas.

`remover_duplicados(df, chaves)` **TRANSFORM**

Remove duplicidades com base nas 3 chaves de identificação da cotação.

`remover_tipo_zero(df)` **TRANSFORM**

Elimina registros sem `TipoCotação` preenchido (marcados como `"0"` na etapa anterior).

`filtrar_condicoes(df)` **TRANSFORM**

Gera um recorte de diagnóstico (não é salvo em arquivo) com linhas de `TipoCotação` vazio e frete/chapa não positivos.

`exportar_excel(df, caminho_saida_base)` **LOAD**

Exporta o DataFrame final para Excel, sem timestamp no nome do arquivo (sobrescreve o arquivo anterior a cada execução).

Chaves de identificação usadas na consolidação

Chave	Coluna original
ID da Cotação SAP	<code>ID da Cotação do SAP</code>
Material	<code>Material: Código do material</code>
Expedição	<code>Cód. Expedição</code>

Estas mesmas três colunas são usadas posteriormente como chave do `LEFT JOIN` na Etapa 3.

ETL_Cotacao_Pipeline.ipynb

Log gerado: pipeline_cotacoes.log

3.1 Objetivo

Extraí a base bruta de cotações comerciais (*COTAÇÕES_CASE*), converte colunas numéricas, padroniza o campo `TipoCotação`, identifica (sem remover) registros duplicados e exporta a base tratada que alimentará a Etapa 3.

ENTRADA

- `COTAÇÕES_CASE.xlsx`
(pasta *Base Dados*, cabeçalho na linha 3 — `header=2`)

SAÍDA

- `Catacao_Result.xlsx`
(pasta *Base Result*)

3.2 Fluxo de execução (função `pipeline_cotacoes()`)

- Carregar** o Excel a partir da 3ª linha (`header=2`), com tipos de coluna pré-definidos.
- Converter colunas decimais** (`Preço Atual`, `Preço Proposto`, `Var. Preço Proposto`) para numérico, transformando erros em `NaN`.
- Tratar `TipoCotação`**: substitui vazio por `"Vazio"`.
- Tratar decimais vazios**: substitui `NaN` /vazio das colunas decimais por `0`.
- Verificar duplicidades** (apenas identifica e loga a quantidade — não remove os registros) com base em `ID da Cotação do SAP` + `Material` + `Cód. Expedição`.
- Exportar** o resultado para `Catacao_Result.xlsx`.

3.3 Funções principais

`carregar_arquivo(caminho, dtype_cols)` **EXTRACT**

Lê o Excel a partir da linha de cabeçalho correta (`header=2`), com tipos string definidos para colunas-chave e categóricas.

`converter_decimal(df, cols)` **TRANSFORM**

Converte `Preço Atual`, `Preço Proposto` e `Var. Preço Proposto` para numérico com `errors="coerce"` (valores inválidos viram `NaN`).

`tratar_tipo_cotacao(df)` **TRANSFORM**

Substitui valores vazios/nulos de `TipoCotação` pelo rótulo `"Vazio"`.

`tratar_decimais_vazios(df, cols)` **TRANSFORM**

Preenche com `0` os valores ausentes das colunas decimais.

`verificar_duplicados(df, chaves)` **TRANSFORM**

Identifica linhas duplicadas pela combinação de chaves e retorna esse subconjunto apenas para fins de log/diagnóstico.

`exportar_excel(df, caminho_saida_base)` **LOAD**

Exporta o DataFrame tratado para `Catacao_Result.xlsx`, sobrescrevendo a cada execução.

Ponto de atenção: diferente do pipeline do Salesforce, este notebook **não remove** as duplicidades encontradas — apenas as reporta no log. Isso significa que registros duplicados podem seguir para a base final se não houver tratamento adicional.

Concolida_Cotacao_SalesForce.ipynb

Depende dos resultados das Etapas 1 e 2

4.1 Objetivo

Consolida as bases tratadas de Cotação e Salesforce (geradas nas etapas anteriores), padroniza e converte a coluna de data de criação da solicitação, realiza o `LEFT JOIN` entre as bases e aplica a regra de negócio de **janela móvel de 30 dias** para calcular indicadores históricos por cliente/material/expedição.

ENTRADA

- `Catacao_Result.xlsx` (saída da Etapa 2)
- `SalesForce_Result.xlsx` (saída da Etapa 1)

SAÍDA

- `BaseFinal_Cotacao_SalesForce.xlsx`
(pasta *Base Result*)

4.2 Fluxo de execução (função `pipeline_etl()`)

- Carregar** os dois arquivos Excel de resultado (`Catacao_Result.xlsx` e `SalesForce_Result.xlsx`).
- Padronizar nomes de colunas:** remove espaços extras e quebras de linha dos cabeçalhos.
- Converter datas:** detecta automaticamente a coluna de criação da solicitação (nome contendo "Criação" e "Sol"), identifica o formato textual e converte para `datetime`, renomeando para `Dt_Criacao_Sol`.
- LEFT JOIN:** une a base de Cotação com colunas selecionadas do Salesforce (`Cód. Expedição`, `Frete Comercial`, `Chapa`), usando as chaves `ID da Cotação do SAP` + `Material` + `Cód. Expedição`. Valores não encontrados em `Frete Comercial` / `Chapa` são preenchidos com `0`.
- Aplicar regras de janela de 30 dias:** calcula, por grupo (`Código do Cliente` + `Material` + `Cód. Expedição`), a quantidade de cotações aprovadas e a soma da variação de desconto nos 30 dias anteriores a cada registro.
- Exportar** o resultado final para `BaseFinal_Cotacao_SalesForce.xlsx`.

4.3 Funções principais

`carregar_excel(caminho)` **EXTRACT**

Carrega o Excel validando previamente se o arquivo existe, lançando erro caso não encontre.

`padronizar_colunas(df)` **TRANSFORM**

Remove espaços em branco e quebras de linha (`\n`, `\r`) dos nomes das colunas.

`detectar_formato_data(valor)` **TRANSFORM**

Identifica, via expressões regulares, o formato textual da data (dd/mm/aaaa, aaaa-mm-dd, com hora, etc.) — usado apenas para diagnóstico.

`converter_datas(df)` **TRANSFORM**

Localiza automaticamente a coluna de data de criação da solicitação e converte para `datetime` (formato dia primeiro — `dayfirst=True`), renomeando-a para `Dt_Criacao_Sol`.

`aplicar_left_join(df_catacao, df_salesforce)` **TRANSFORM**

Renomeia `Material: Código do material` para `Material` na base Salesforce e realiza o `LEFT JOIN` com a base de Cotação pelas 3 chaves-padrão.

`calcular_janela_30d(grupo)` **TRANSFORM**

Para cada grupo (cliente/material/expedição), ordena por data e calcula a soma acumulada em janela móvel de 30 dias, usando o valor do registro *anterior* (`shift(1)`) para não incluir o próprio registro no cálculo.

`aplicar_regras_janela(df)` **TRANSFORM**

Define quais cotações são "elegíveis" (Aprovadas e do tipo Banda/PrecoFixo), agrupa os dados e aplica `calcular_janela_30d` a cada grupo.

`exportar(df)` **LOAD**

Remove a coluna auxiliar de diagnóstico `Formato Detectado` e exporta o resultado final para `BaseFinal_Cotacao_SalesForce.xlsx`.

4.4 Regra da janela móvel de 30 dias (detalhe)

Critério de elegibilidade — uma cotação é considerada "elegível" quando, simultaneamente:

- `Status da Cotação` = "Aprovada"
- `TipoCotação` está entre `"Banda"` ou `"PrecoFixo"`

Coluna calculada	Descrição
<code>Qtd_cotacoes_aprovadas_30d</code>	Contagem de cotações elegíveis nos 30 dias anteriores ao registro atual (por cliente/material/expedição).
<code>Soma_variacao_desconto_30d</code>	Soma da variação de preço proposto (<code>Var. Preço Proposto</code>) das cotações elegíveis nos 30 dias anteriores.

Detalhe técnico importante: o cálculo usa `shift(1)` antes de aplicar o `rolling("30D")` . Isso significa que o próprio registro do dia não entra na soma — apenas os registros elegíveis anteriores a ele dentro da janela de 30 dias, evitando "vazamento" do valor atual para o próprio indicador histórico.

5. Observações técnicas e pontos de atenção

Bibliotecas utilizadas por notebook

Biblioteca	Uso	Notebooks
pandas	Leitura/escrita de Excel, manipulação de DataFrames, merge, rolling window	Todos os 3 notebooks
os	Verificação de existência de arquivo e composição de caminhos	Concolida_Cotacao_SalesForce
re	Expressões regulares para detecção do formato textual das datas	Concolida_Cotacao_SalesForce
logging	Registro de logs de execução em arquivo (<code>.log</code>)	ETL_SalesForce_Pipeline, ETL_Cotacao_Pipeline
datetime	Manipulação de datas e intervalos de tempo (<code>timedelta</code>)	Todos os 3 notebooks

Caminhos de arquivo fixos (hardcoded)

Os três notebooks utilizam caminhos absolutos do Windows fixos no código (ex.: `C:\Users\stenz\Documents\InterCement\...`). Caso o processo seja executado em outra máquina ou usuário, os caminhos em `PASTA_BASE`, `caminho` e `caminho_saida` precisam ser ajustados manualmente em cada notebook.

Notebook	Variável	Caminho configurado
ETL_SalesForce_Pipeline	caminho	...\Base Dados\SalesForce_CASE.xlsx
ETL_SalesForce_Pipeline	caminho_saida	...\Base Result\SalesForce_Result.xlsx
ETL_Cotacao_Pipeline	caminho	...\Base Dados\COTAÇÕES_CASE.xlsx
ETL_Cotacao_Pipeline	caminho_saida	...\Base Result\Catacao_Result.xlsx
Concolida_Cotacao_SalesForce	PASTA_BASE	...\Base Result

Resumo de riscos identificados

Duplicidades não removidas na base de Cotação — o notebook `ETL_Cotacao_Pipeline` apenas identifica duplicidades, mas não as remove antes de exportar; elas podem se propagar até a base final.

Dependência de execução manual e sequencial — não há orquestração automática entre os três notebooks; a ordem correta (SalesForce → Cotação → Consolidação) deve ser garantida manualmente pelo usuário.

Caminhos fixos por usuário — os caminhos de pastas usam o usuário `stenz`, exigindo ajuste em caso de migração de ambiente.

Detecção automática de coluna de data — a função `converter_datas` localiza a coluna de data por nome (contendo "Criação" e "Sol"); alterações no nome da coluna de origem podem quebrar o processo.